

sg-vault PKI Workflows and Operating Modes

version v0.13.32 **date** 13 Mar 2026 **role** Architect **status** Companion to architecture spec — PKI deep dive **context** The architecture spec (v0.13.30) defines branch-level signing as mandatory. This document extends the PKI model to cover user-level signing, the full spectrum of operating modes, and the provenance guarantees each mode provides. The fundamental rule: only public keys enter the vault. User private keys are managed entirely outside the vault.

1. The Problem This Document Solves

The architecture spec establishes that every commit is signed by a **branch key**. This proves device provenance ("this commit came from this specific clone"). But it does not prove person identity ("this commit was made by Dinis Cruz").

In many workflows, device provenance is sufficient. In others — audit trails, regulatory compliance, multi-party collaboration, investor-facing data rooms — you need person identity, and sometimes you need both.

This document maps the full spectrum: from anonymous single-user vaults to multi-party signed collaboration, showing exactly which keys are in play at each level and what provenance guarantees they provide.

2. The Four Keys

The vault system has four types of cryptographic key, each serving a different purpose. Not all are present in every operating mode.

1. VAULT KEY (symmetric, derived)

What: read_key (AES-256) + write_key (HMAC)

From: HKDF-SHA256 of vault passphrase + vault_id

Proves: "I have access to this vault"

Scope: the vault as a whole

2. BRANCH KEY (EC P-256, per clone)

What: key pair generated on clone

From: random, on the device at clone time

Proves: "this commit came from this specific clone/device"

Scope: one clone, one device, one session lineage

3. NAMED BRANCH KEY (EC P-256, per named branch)

What: key pair for the named branch (e.g. "current")

From: generated at init, stored encrypted in vault

Proves: "this merge was performed by an authorised entity"

Scope: the named branch

4. USER KEY (EC P-256 or RSA-4096, per person/entity)

What: the user's personal key pair

From: sg-send-cli pki keygen (already exists in CLI)

Proves: "this action was performed by this specific person"

Scope: the person, across all devices, all vaults, all time

CRITICAL: only the PUBLIC key is added to the vault (for verification by other participants). The PRIVATE key is managed entirely outside the vault — in the user's own key store (~/.sg-send/keys/), OS keychain, hardware token, or a separate vault they control. The vault being signed into NEVER holds the user's private key. This is a fundamental separation: the vault trusts the signature, not the key.

Keys 1–3 are in the current architecture spec. Key 4 (user key) exists in the CLI already (`pki keygen`), (`pki sign/verify`), (`pki encrypt/decrypt`) but is not yet connected to the commit model.

3. The Operating Modes

Each mode adds keys to the system. More keys = stronger provenance guarantees.

Less keys = simpler workflow, lower friction.

The modes are not exclusive — they're additive. A vault can operate at any level, and different users within the same vault can operate at different levels.

Mode 1: Anonymous Vault (Vault Key Only)

Keys in play: vault key (read_key + write_key)
Keys absent: no branch keys, no user keys
Provenance: NONE — "someone with the vault key did this"

This is the **current v0.5.x behaviour**. Push overwrites the remote. No branches, no signatures, no identity. The vault key is the only secret.

Useful for: personal scratch vaults, config stores, single-user workflows where provenance doesn't matter.

Commit:
tree_id: obj-`{hash}`
parents: [obj-`{hash}`]
timestamp: 1710244800000
message: "update config"
branch_id: null ← no branch
signature: null ← no signature
author: null ← no author

What you can prove: nothing about who did what. Only that someone with the vault key made changes.

Mode 2: Device-Provenance Vault (+ Branch Keys)

Keys in play: vault key + branch keys (per clone)
Keys absent: no user keys
Provenance: DEVICE — "this commit came from Device 1's clone"

This is what the **architecture spec mandates**. Every clone gets a branch key. Every commit is signed by the branch key. The named branch has its own key for merge signatures.

Useful for: multi-device workflows, AI agent collaboration, team vaults where you need to know **WHICH** device/agent made each change but don't need to know **WHO** the human behind it is.

Commit:
tree_id: obj-`{hash}`
parents: [obj-`{hash}`]
timestamp: 1710244800000
message: "update config"
branch_id: branch-clone-p4n8 ← identifies the clone
signature: SIGN(branch_priv) ← proves device provenance
author: null ← no person identity

What you can prove:

- This commit was made by the same device as commit X (same branch key)
- This commit was NOT made by Device 2 (different branch key)
- The commit chain has not been tampered with (signatures verify)
- The merge into the named branch was authorised (named branch key signed it)

What you cannot prove:

- Which human was sitting at the device
- Whether the same person was behind two different device branches

Mode 3: Author-Identified Vault (+ User Keys)

Keys in play: vault key + branch keys + user keys

Keys absent: none

Provenance: PERSON — "Dinis Cruz made this commit from Device 1"

This is the **full model**. The user configures their personal key pair (via `pki keygen`) or by importing an existing key). Commits carry both the branch signature (device) AND the user signature (person).

Useful for: audit trails, regulatory compliance, data rooms, investor-facing vaults, multi-party collaboration where each person's actions must be individually attributable.

Commit:

```
tree_id:      obj-{hash}
parents:      [obj-{hash}]
timestamp:    1710244800000
message:      "update config"
branch_id:    branch-clone-p4n8  ← identifies the clone
signature:    SIGN(branch_priv)  ← proves device provenance
author_key_id: key-{user_key_id}  ← identifies the person
author_signature: SIGN(user_priv) ← proves person identity
```

What you can prove:

- Everything from Mode 2, plus:
- Dinis Cruz made this commit (user signature verifies against his public key)
- Dinis Cruz was using Device 1 at the time (branch signature + user signature)
- Even if someone stole Device 1, they couldn't forge Dinis's user signature (user private key is separate from branch private key)

What you cannot prove:

- That Dinis actually reviewed the content (only that his key signed it)
- Non-repudiation in a legal sense (depends on jurisdiction and key management)

Mode 4: Countersigned Vault (+ Third-Party Attestation)

Keys in play: vault key + branch keys + user keys + external attestation keys

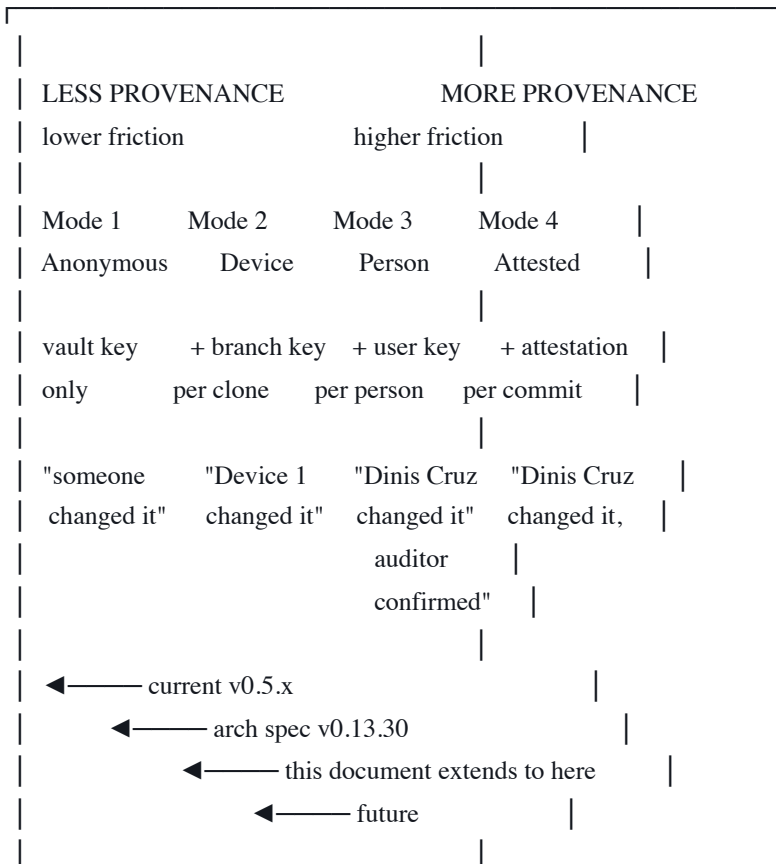
Provenance: ATTESTED — "Dinis Cruz made this, and Auditor X confirmed it"

This is a **future mode** — not for initial implementation. The commit carries an additional countersignature from an external party (auditor, compliance system, notary service).

Commit:

```
...
signature:    SIGN(branch_priv)
author_key_id: key-{user_key_id}
author_signature: SIGN(user_priv)
attestations: [
  {
    attester_key_id: key-{auditor_key_id}
    attester_signature: SIGN(auditor_priv)
    attested_at:    1710250000000
    scope:          "content_review"
  }
]
```

4. The Provenance Spectrum



Each vault chooses its operating mode. Different users within the same vault can operate at different levels — a user who hasn't configured a personal key makes commits at Mode 2 (device provenance only), while a user who has configured one makes commits at Mode 3 (person identity).

The vault's **minimum provenance level** can be enforced via a policy field in the vault metadata:

```
json
{
  "schema": "vault_policy_v1",
  "minimum_provenance": "device",
  "require_author_signature": false,
  "require_attestation": false
}
```

When `(require_author_signature: true)`, the CLI refuses to create commits without a configured user key. This is a policy decision, not a cryptographic enforcement — it's checked client-side. The named branch merge can additionally verify author signatures on incoming commits before accepting the merge.

5. User Key Lifecycle

5.1 Key Generation

The user key pair already exists in the CLI:

```
$ sg-send-cli pki keygen --label "Dinis Cruz"
generates: EC P-256 signing key + RSA-4096 encryption key
stored in: ~/.sg-send/keys/{fingerprint}/
```

5.2 Connecting User Key to Vault

When a user wants to sign commits with their identity:

```
$ sg-send-cli config set author-key <fingerprint>
stores: fingerprint in local/author_key_fingerprint
from now on: every commit includes author_key_id + author_signature
```

This is per-clone config (stored in `(local/)`). The user can have different author keys on different devices (e.g. a work key and a personal key), or the same key imported to multiple devices.

Only the public key enters the vault. When author signing is configured, the CLI publishes the user's public key to `(bare/keys/key-{id})` so other vault participants can verify signatures. The private key stays in `(~/.sg-send/keys/)` on the user's own machine — it is NEVER written to the vault, NEVER synced to the server, NEVER accessible to other vault participants. The vault holds verification material only.

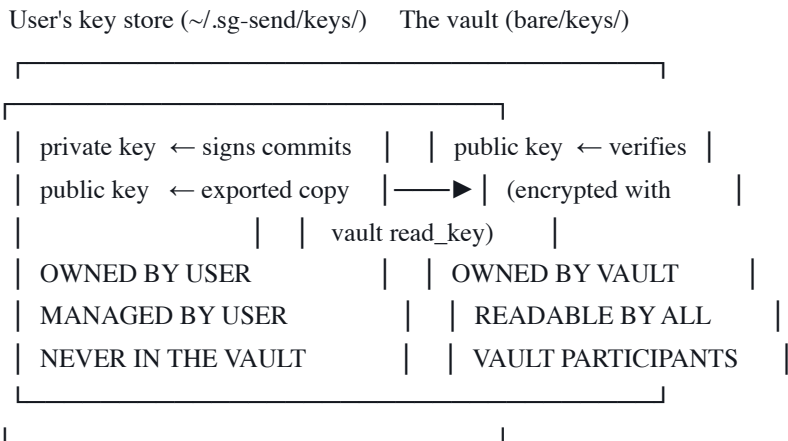
5.3 Publishing User Public Key

For other vault participants to verify author signatures, the user's public key must be accessible. The vault stores **ONLY** the public key:

```
bare/keys/key-{{user_pub_id}} = ENC(read_key, user_public_key)
```

This is a one-way publication: public key goes **IN** to the vault for verification.

Private key **NEVER** enters the vault. The relationship is:



Additionally, the user can publish their public key to external registries (SG/Send PKI registry at `/keys/publish`, personal website, key server) for cross-vault identity verification. But the vault itself only needs the public key.

5.4 User Key vs Branch Key — Why Both?

Branch key:

- ✓ Proves device provenance
- ✓ Unique to this clone (never on another device)
- ✓ Private key in local/ (device-bound)
- ✓ Lost = branch locked (normal lifecycle)
- ✗ Can't prove which person used the device
- ✗ Can't link activity across devices

User key:

- ✓ Proves person identity
- ✓ Travels across devices (same person, many clones)
- ✓ Links activity across vaults and time
- ✓ Private key **OUTSIDE** the vault (user-managed, ~/.sg-send/keys/)
- ✓ Only public key in vault (for verification only)
- ✗ Can't prove which specific device was used
- ✗ If compromised, all past signatures are suspect

Both together:

- ✓ "Dinis Cruz (user key) was on his laptop (branch key) at this time"
- ✓ Even if someone steals the laptop, they can't forge the user signature (user private key is **NOT** on the laptop's vault clone — it's in ~/.sg-send/)

✓ Even if someone compromises the user key, the branch signature still proves which device was involved

The two signatures are **complementary**. The branch key is the "what device" channel. The user key is the "which person" channel. Neither alone provides both signals.

6. PKI Workflows in Practice

6.1 Solo Developer (Mode 2 — Device Provenance)

The default. No user key configured. Branch keys auto-generated on clone.

```
$ sg-send-cli init my-vault
$ echo "hello" > README.md
$ sg-send-cli commit -m "first file"
$ sg-send-cli push origin
```

Commit signed by: branch-clone-p4n8 (device key)
Author: (none configured)

6.2 Team with Person Identity (Mode 3)

Each team member configures their user key.

```
[Dinis — Device 1]
$ sg-send-cli pki keygen --label "Dinis Cruz"
$ sg-send-cli config set author-key fp_dinis_a1b2
$ sg-send-cli commit -m "architecture update"
```

Commit signed by: branch-clone-p4n8 (Dinis's laptop)
Author signed by: key-fp_dinis_a1b2 (Dinis Cruz)

```
[Agent — Lambda]
$ sg-send-cli commit -m "billing event batch"
```

Commit signed by: branch-clone-q9w3 (Lambda agent)
Author: (none — agents don't have personal keys by default)

The `log --graph` output shows both signatures:

```
$ sg-send-cli log --graph --show-author

* obj-h_cmt_m003 (current) Merge into current
| branch: branch-named-x7k2
|
| * obj-h_cmt_0003 architecture update
| branch: branch-clone-p4n8
| author: Dinis Cruz (key-fp_dinis_a1b2) ✓
|
| * obj-h_cmt_b001 billing event batch
| branch: branch-clone-q9w3
| author: (none)
```

6.3 Data Room with Mandatory Author Signing (Mode 3, Enforced)

Vault policy requires author signatures on all commits.

```
Vault policy:
require_author_signature: true

[User without configured author key]
$ sg-send-cli commit -m "update"
Error: this vault requires author signatures.
    Run 'sg-send-cli config set author-key <fingerprint>'
    to configure your personal key.

[User with configured author key]
$ sg-send-cli commit -m "update"
✓ Signed by branch-clone-p4n8 + author key-fp_dinis_a1b2
```

6.4 Cross-Vault Identity

A user's key pair works across vaults. If Dinis publishes his public key to the SG/Send PKI registry, anyone can verify his commits in any vault:

```
$ sg-send-cli pki verify-author obj-h_cmt_0003
Branch: branch-clone-p4n8 ✓ (device provenance)
Author: key-fp_dinis_a1b2 ✓ (person identity)
Name:   Dinis Cruz
Source: vault key registry + SG/Send PKI registry

$ sg-send-cli pki verify-author obj-h_cmt_b001
Branch: branch-clone-q9w3 ✓ (device provenance)
Author: (not signed — no author key configured)
```

6.5 Agent Identity (Mode 2 with Metadata)

AI agents don't have "personal keys" in the human sense, but they can carry identity metadata in their branch:

```
branch-clone-{agent_id} = ENC(read_key, {  
  name: "billing-agent-v3",  
  type: "clone",  
  agent_model: "claude-sonnet-4-20250514",  
  agent_session: "session_01abc...",  
  created_at: 1710244800000  
})
```

The branch metadata identifies the agent. The branch key proves the same agent instance across commits. This is Mode 2 (device provenance) with rich metadata — the agent's "device" is its session/environment.

6.6 Change Pack Creator Identity

Change packs (keyless creators) carry their own identity model:

```
{  
  "schema": "change_pack_v1",  
  "creator_key": "<submitter_fingerprint>",  
  "signature": "<SIGN(creator_priv, payload_hash)>"  
}
```

The `creator_key` identifies the external system that submitted the change pack. GC verifies the signature on drain. The creator's public key can be pre-registered in the vault's key registry or resolved externally.

This is a distinct identity from branch keys and user keys — the change pack creator may not have a branch (no clone) and may not be a person (no user key). They have only a signing key and a scoped write token.

7. Signature Verification Depth

Different use cases require different verification depths. The CLI supports multiple levels:

Level 0: No verification	
"I trust the remote"	
Pull and use without checking any signatures.	
Fastest. Appropriate for personal vaults.	
Level 1: Branch signature verification	
"I trust that commits came from the branch they claim"	
Verify branch key signed each commit.	
Detects: tampered commits, forged branch attribution.	
Default for most workflows.	
Level 2: Author signature verification	
"I trust that the claimed person actually made this commit"	
Verify user key signed each commit that claims an author.	
Detects: impersonation, stolen device (without user key).	
Recommended for regulated data rooms.	
Level 3: Full chain verification	
"I trust the entire history from root to HEAD"	
Walk the full commit chain, verify every signature,	
confirm no gaps, no orphaned parents, no missing keys.	
Detects: history rewriting, cherry-picked commits.	
For audit and compliance.	
Level 4: Attestation verification (future)	
"I trust that an external party confirmed each commit"	
Verify countersignatures from auditors/notaries.	
For legal and regulatory evidence chains.	

Verification level is a client-side setting:

```
$ sg-send-cli config set verification-level 2
```

The server never verifies signatures (zero-knowledge — it can't read them).

8. Commit Schema — Reserved Fields

To avoid migration pain, the commit schema should reserve fields for all modes from the start, even if initially unused:

```
json
```

```

{
  "schema": "commit_v1",
  "tree_id": "obj-{hash}",
  "parents": ["obj-{hash}"],
  "timestamp": 1710244800000,
  "message": "add initial files",

  "branch_id": "branch-clone-{id}",
  "signature": "<SIGN(branch_key, ...)>",

  "author_key_id": null,
  "author_signature": null,

  "attestations": []
}

```

Fields that are `null` or `[]` add negligible storage cost but prevent a schema migration when the feature is activated. The branch signature remains mandatory. Author and attestation signatures are optional.

Important: the signature covers ALL fields including the reserved ones. This means a commit created with `author_key_id: null` cannot later have an author signature added — the original signature would fail verification. This is intentional: the commit's provenance is immutable at creation time.

9. Key Storage Summary

Key	Where	Travels?
vault read_key	derived from passphrase	anyone with vault key
vault write_key	derived from passphrase	anyone with vault key
clone branch pub	bare/keys/key-{id}	syncs everywhere
clone branch priv	local/branch_key.pem	NEVER leaves device
named branch pub	bare/keys/key-{id}	syncs everywhere
named branch priv	bare/keys/key-{id} (enc)	in vault (encrypted)
user pub (signing)	bare/keys/key-{id}	vault (verify only)
user priv (signing)	~/.sg-send/keys/{fp}/ (user-managed, external)	NEVER IN THE VAULT user's devices only
user pub (encrypt)	bare/keys/key-{id}	vault (verify only)
user priv (encrypt)	~/.sg-send/keys/{fp}/ (user-managed, external)	NEVER IN THE VAULT user's devices only
scoped write token	external system config	given to creators
creator signing key	external system	owned by creator

The vault holds THREE types of private key:

1. Named branch private key (in vault, encrypted — authorises merges)
2. Clone branch private key (in local/, never synced — proves device)
3. User private key — NOT IN THE VAULT. EVER. Managed externally.

The vault holds public keys for verification:

- Branch public keys (clone + named)
- User public keys (for author signature verification)

10. Implementation Phasing

Now (Architecture Spec v0.13.30)

- Mode 2 (device provenance): mandatory branch signing
- Branch key generation on clone
- Named branch key for merge signing
- Commit schema with reserved `author_key_id`, `author_signature`, `attestations` fields (all null)

Next (When Team or Customer Requires Person Identity)

- Mode 3 (author signing): optional user key on commits
- `sg-send-cli config set author-key` command
- User public key published to vault `bare/keys/`
- `log --show-author` flag
- Verification levels 0–3

Future (When Regulated Data Rooms Require It)

- Mode 4 (attestation): countersignatures from external parties
- Vault policy enforcement (`require_author_signature`, `require_attestation`)
- Full chain verification (Level 3)
- Attestation verification (Level 4)
- Integration with external PKI registries

11. Open Questions for Future Work

Question	Context	When to Decide
Should agents have "user keys"?	An AI agent could have a persistent identity key that survives session restarts. Different from a branch key (which is per-clone).	When agent-to-agent collaboration patterns emerge
Key rotation for user keys	If a user rotates their key, old commits have the old signature. Should there be a "key succession" record?	When long-lived vaults with key rotation become common
Revocation	Can a user key be revoked? What happens to commits signed with a revoked key?	When compliance requirements demand it
Delegation	Can a user delegate signing authority to another user or agent? ("Sign on my behalf for the next 24 hours")	When workflow automation requires it
Group keys	Can a team have a shared key that any member can sign with? Different from quorum (which requires N of M).	When team-level attribution is needed alongside individual
Hardware key support	User private keys on hardware tokens (YubiKey, TPM). The CLI would call an external signing interface.	When security-sensitive deployments require it

SGraph Send — Architect — v0.13.32 PKI workflows and operating modes Companion to architecture spec — extends branch signing to full provenance spectrum Principle: vault holds public keys for verification. User private keys NEVER enter the vault.